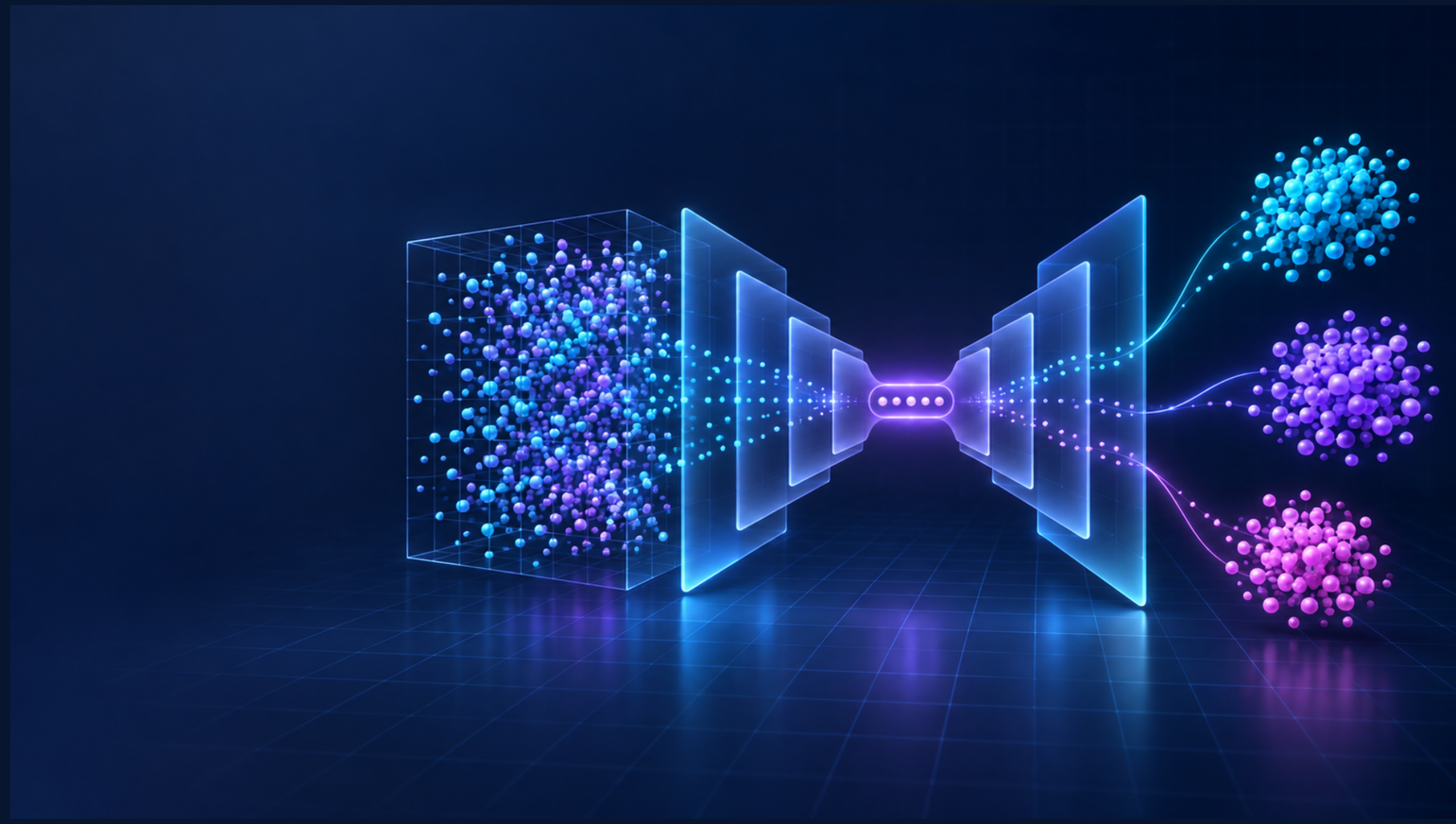
CMPE 255 | DATA MINING SURVEY

Autoencoder-Based Deep Clustering



How representation learning turns high-dimensional data into clustering-friendly structure

Based on Leiber, Miklautz, Plant & Böhm (arXiv:2504.02087v2)

The data mining problem

Traditional clustering sees the raw feature space. Deep clustering learns a space where the grouping task becomes easier.

High-dimensional input

Images, text, tabular records

Learned representation

A compact nonlinear embedding

Cluster structure

Compact groups and clearer boundaries

The survey treats autoencoders as a reusable sandbox: the encoder changes the geometry, while the clustering objective decides what “useful” means.

One taxonomy, three training loops

The key distinction is how representation learning and clustering influence each other.

Sequential

Train AE, then cluster embedding

Simple and modular. The embedding may ignore the task.

Alternating

Update embedding and clusters in turn

Representation and labels refine each other.

Simultaneous

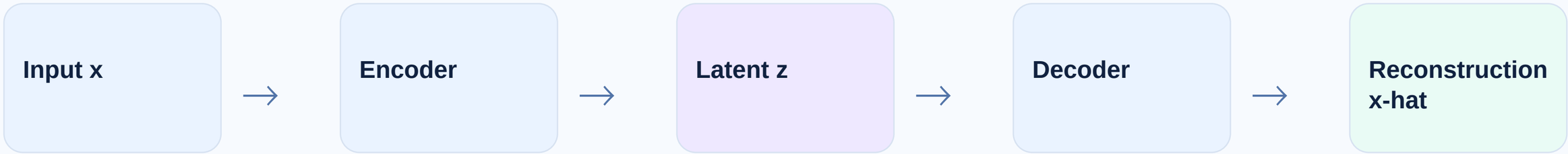
Optimize reconstruction and clustering together

Joint objective with tighter coupling.

Representative methods include AE + k-Means, N2D, DEC, IDEC, DCEC, DCN and DKM.

The autoencoder is the representation engine

An encoder compresses x into z. A decoder reconstructs x from z. The bottleneck makes the latent space explicit.



A common fully connected backbone uses a mirrored encoder and decoder with a low-dimensional bottleneck. Convolutional variants fit image structure better.

A low reconstruction error does not guarantee clusters that match the downstream task.

Loss design controls the trade-off

Deep clustering methods add a clustering signal to reconstruction. The balance changes what the latent space preserves.

Reconstruction loss

Keeps decoded samples close to the input.

Useful when preserving information matters.

Clustering loss

Pulls the embedding toward compact, separable or task-specific groups.

Total objective = reconstruction term + weighted clustering term

The weight is a modeling decision, not a detail.

What the methods gain, and what they give up

AE + k-Means	Easy baseline	No task feedback after embedding
N2D	Uses a learned manifold	New-sample prediction is awkward
DEC / IDEC	Shapes latent space	Sensitive to initialization and loss balance
DCEC	Convolutional bias	More tied to image-like inputs
DKM	Deep k-Means objective	Requires careful optimization choices

The strongest contribution is the shared design space. Many methods differ less in the encoder than in how clustering evidence feeds back into training.

Metrics can reward different stories

A credible comparison needs more than one score. External labels, internal geometry and stability answer different questions.

External	**ARI, NMI, ACC**	Do groups align with known labels?
Internal	**Silhouette, WCSS**	Are points compact and separated?
Robustness	**Seeds, perturbations**	Does the solution survive randomness?
Practical	**Inference, scalability**	Can the model label new samples?

Ablation question: how much of the gain comes from the autoencoder, clustering term, augmentation or optimizer?

Where the survey points next

Representation quality

Separate semantic structure from nuisance variation?

Unknown cluster count

Decide k without labels?

Shift and deployment

Keep clusters meaningful as data moves?

Evaluation practice

Report stability and compute cost consistently?

The data mining lesson is broader than one algorithm: representation learning changes the space in which similarity is defined.

Takeaways for a data mining practitioner

1. Start with AE + k-Means as a transparent baseline
2. Add clustering feedback only when the baseline exposes a real failure
3. Report metrics, seeds, ablations and deployment behavior together
4. **Treat the latent space as a modeling decision, not a visualization afterthought**

Source: Leiber et al., “An Introductory Survey to Autoencoder-based Deep Clustering,” arXiv:2504.02087v2, 2025.